Promise App POC Demo Walkthrough

Purpose: show that a phone signs and durably queues exact message bytes, a relay stores and routes those bytes, and a separate computer recipient independently verifies them. This is a POC—not a claim of recipient acceptance, production security, or a universal PromiseGrid transport.

> **Keep private:** relay admin token, private computer identity file, and generated local evidence. Do not put them in a slide deck, QR code, chat, or repository.

1. What the demo proves

Visible result	What it proves	What it does not prove
`Relay check: ready`	The phone can reach the selected relay health endpoint.	That a message was sent or received.
Visible message CID; phone outbox is queued	The phone durably saved exact signed bytes before upload.	That the relay or recipient has seen them.
`Relay accepted <CID>`	The relay accepted the phone's exact bytes for that CID.	Recipient verification or human acceptance.
`RECIPIENT VERIFIED cid= <CID>`	The computer retrieved bytes and independently checked their CID, POC protocol selector, Grid-CBOR structure, signature, and recipient binding.	Human acceptance, execution, or trust in the promise.

2. Before the audience arrives

1. Connect the phone to the same private Wi-Fi as this computer. Keep a USB cable available as a fallback.
2. Choose an unused port, such as `18081`: `ss -ltnp '( sport = :18081 )'`.
3. Find the computer's private LAN address with `hostname -I`. Use an address reachable by the phone, for example `10.0.99.219`.
4. Build and install the current app before the demo: `GRADLE_USER_HOME=.local/gradle ./scripts/build-phone-computer-host.sh`, then `adb install -r android/app/build/outputs/apk/debug/app-debug.apk`.

3. Start the private relay

In terminal A, substitute your private LAN address and chosen port. Leave the terminal open.

```
export PROMISE_ADMIN_TOKEN="$(openssl rand -hex 32)"
export PROMISE_ADVERTISE_URL=http://10.0.99.219:18081
export PROMISE_LISTEN_ADDR=0.0.0.0:18081
./scripts/start-lan-relay.sh
```

In terminal B, verify host health:

```
curl --fail --silent --show-error http://127.0.0.1:18081/healthz
# Expected: {"status":"ok"}
```

> Before presenting the app, open `http://<LAN-IP>:<PORT>/healthz` in the phone browser. It must return `{"status":"ok"}`. If it does not, resolve Wi-Fi/client-isolation/firewall access first.

4. Prepare the computer recipient

```
make peer-id
make peer-qr-open
```

The QR contains only the computer recipient's public CID—never a private key, relay token, or message. Leave it visible on the computer screen.

5. Phone walkthrough

1. Open **Promise App**.
2. Tap **Scan relay URL QR** and scan a QR that encodes `http://10.0.99.219:18081`. This QR configures transport; it is not a recipient.
3. Tap **Check relay** and show `Relay check: ready`.
4. Tap **Scan recipient identity QR**, scan the computer recipient QR, and show the populated recipient CID. This QR selects the recipient; it is not a relay URL.
5. Enter a harmless body, such as `POC demo test`.
6. Tap **Sign, queue, and upload**.
7. Show the visible **Message CID** and `Relay accepted <CID>`. Use **Copy message CID** if needed at the terminal.

> **Presenter line:** “The phone saved the signed message and the relay accepted those exact bytes. That is not yet recipient verification.”

6. Independent computer verification

Replace `<CID>` with the CID shown by the phone:

```
cd backend
go run ./cmd/promise-peer inbox \
  -relay http://10.0.99.219:18081 \
  -identity-file ../.local/promise-peer/identity.ed25519 \
  -inbox-dir ../.local/promise-peer/inbox \
  -expect-cid <CID>
```

Show `RECIPIENT VERIFIED cid=<CID>` and compare it to the phone's visible CID.

> **Presenter line:** “A separate recipient fetched the bytes and checked the CID, protocol profile, signature, and intended recipient. Verification is still not human acceptance.”

7. Repeatable checks

```
# Attached-phone health check over Wi-Fi; USB=1 also checks a USB reverse route.
make phone-smoke RELAY=http://10.0.99.219:18081
make phone-smoke RELAY=http://10.0.99.219:18081 USB=1

# Validate host tools, APK, loopback health, and advertised-LAN health.
./scripts/validate-phone-computer-host.sh \
  -relay http://10.0.99.219:18081 \
  -apk android/app/build/outputs/apk/debug/app-debug.apk \
  --check-relay
```

8. USB fallback

Use this only when the phone is connected, unlocked, authorized for USB debugging, and Wi-Fi cannot reach the host. It is a host-specific demo transport, not a protocol rule.

```
adb reverse tcp:18081 tcp:18081
adb reverse --list
adb shell 'curl --fail --silent http://127.0.0.1:18081/healthz'
```

Set the phone's relay URL to `http://127.0.0.1:18081`, save it, and check the relay.

9. Fast troubleshooting

Symptom	Check and resolution
Phone says relay failed	Open `http://<LAN-IP>:<PORT>/healthz` in the phone browser. It must return `{"status":"ok"}`. Otherwise check same Wi-Fi, client isolation, host firewall, selected port, and relay process.
Port already in use	Run `ss -ltnp '( sport = :PORT )'`. Choose a free port and use it consistently in listener, phone URL, and verifier command.
QR scan fails	Keep the QR visible at high brightness. The app recipient field is a fallback; never include a private key or relay token in QR content.
Verifier cannot find the CID	Compare the phone relay URL, recipient CID, and visible Message CID. Run the verifier with that exact CID; relay status alone is not evidence of recipient verification.
Need a clean phone	Android Settings → Apps → Promise App → Storage & cache → Clear storage. This removes saved settings, outbox, and the app device identity.

10. Close the demo

1. Keep the phone Message CID and computer `RECIPIENT VERIFIED` line together as evidence.
2. Stop the relay with `Ctrl-C` in terminal A.
3. Do not publish `.local/`; it contains local relay data, identities, and evidence.